

Technē: A Deterministic Craft Layer for Model Context Protocol Servers, Grounded in LLM-Authored X3D

Anonymous Author(s)

ABSTRACT

When a language model builds 3D content by calling tools through the Model Context Protocol (MCP), the JSON Schema ensures each call is well-formed—but not that the result will render correctly. A texture placed in the wrong slot vanishes. A humanoid skeleton routed to the wrong parent field is invisible. The schema passes; the pixels fail. We present Technē, a deterministic MCP proxy for the Web3D x3d_mcp server that catches these failures. Sitting between the model and the real server, Technē intercepts each tool call and (1) repairs sloppy arguments using Postel-style coercion, (2) checks a catalog of documented X3D silent-failure modes, returning prescriptive corrections that name the exact fix, and (3) renders the scene and checks for blank frames—a test no schema validator can perform. An opt-in provenance gate extends the same approach to content honesty. On correct input Technē is invisible. The system runs no LLM in the enforcement loop, carries 147 passing unit tests, and is measured against the bare server in a controlled comparison.

CCS CONCEPTS

• **Computing methodologies** → **Intelligent agents**; *Computer graphics*; • **Software and its engineering** → **Software verification and validation**; • **Information systems** → *Web services*; • **Human-centered computing** → *Human computer interaction (HCI)*.

KEYWORDS

Model Context Protocol, LLM tool use, agentic systems, Schema-Aligned Parsing, BAML, X3D, HAnim, Web3D, deterministic validation, prescriptive correction, verify-by-render, occupation gate, man-in-the-middle proxy, data provenance, documented vs interpretive

1 INTRODUCTION

A language model building an X3D scene creates a Physical-Material, creates an ImageTexture for a brick surface, and attaches the texture to the material. Each tool call is well-formed. The schema validator reports success. But the texture landed in its default slot—texture—which a physically based material does not define. It should have been baseTexture. The brick vanishes. No error is thrown.

This class of failure recurs across X3D [4] authored through the Model Context Protocol (MCP) [1]. A humanoid skeleton [5] routed to children instead of skeleton renders invisible. Mismatched interpolator arrays corrupt an animation silently. An EnvironmentLight whose global flag is dropped by the serializer kills image-based lighting without a trace. In every case the schema says the scene is correct. The failure shows only when someone renders and looks.



Figure 1: Technē as an MCP man-in-the-middle. Each tool call is repaired, validated against craft rules, and either forwarded with corrected arguments or blocked with a prescriptive correction naming the fix. The occupation gate renders and inspects the result.

The gap is between two kinds of knowledge. The schema encodes what we might call *epistēmē*: the formal structure—field names, types, legal values—decidable, closed, complete on its own terms. What it does not encode is *technē*: the craft of making the thing render correctly—which slot to use, which defaults to override, which orders to respect. A desktop authoring tool supplies this craft through its interface and enforced workflows. X3D-the-format carries only *epistēmē* and assumes the *technē* lives in the authoring tool. When the author is a language model emitting raw tool calls, the craft has nowhere to live.

A digital-content-creation application—Cinema 4D, Blender, X3D-Edit—supplies this craft implicitly, through an interface that enforces which child attaches to which parent field, a palette that presents only the legal texture slots, and a scene graph that maintains DEF/USE ordering by construction. The craft is invisible because the tool makes it impossible to violate. X3D-the-format carries only the schema and assumes the authoring tool handles the rest. When the author is a language model emitting raw arguments through MCP, that assumption fails.

This paper gives it somewhere to live. Technē is a man-in-the-middle proxy on the MCP transport (Fig. 1). The model connects to Technē instead of the real x3d_mcp server [10]; Technē launches the server as a subprocess and intercepts every tool call. On correct input it is invisible—same tools, same arguments, same results. On input that would fail silently, it either repairs the arguments and forwards the corrected call, or blocks the call and returns a prescriptive correction—an error message that *is* the fix, not a bare “invalid.”

Built on BAML [2] and grounded in LLM-authored X3D, the system makes four contributions:

- (1) **In-transport argument repair.** BAML’s Schema-Aligned Parsing (SAP) repairs model output where application code calls a model (the app→model seam) [2]. Technē inverts the direction: it applies SAP-style repair to the arguments a model emits *into* a tool, intercepted on the MCP wire (model→tool). Productized SAP occupies the app-to-model seam; Technē is the first, to our knowledge, to point this discipline at tool-call arguments mid-flight in an MCP proxy.

- (2) **Prescriptive corrections from a documented catalog.** The known X3D silent-failure modes—compiled from the `x3d.py` bug report [10]—are recast as a rule→fix table where the error message names the fix (“use `containerField='baseTexture'`”), not a bare “invalid.” The diagnostic a violation produces is a *prescriptive correction*, and the craft holds mechanically whether or not the model understood it.
- (3) **An occupation gate that renders and looks.** After validation, the gate executes a renderer and checks the pixels for blank frames—a test no schema validator or assertion language can perform, since both operate only on parsed values, not on the side effects of running those values through a renderer. The gate is graduated by irreversibility: automatic for cheap reversible work, requiring human sign-off for expensive irreversible work.
- (4) **A provenance bridge (a scope extension).** Beyond X3D correctness, the same approach—documented discipline compiled into a deterministic check—extends to content honesty. An opt-in gate enforces *documented/interpretive/generated* labeling against an asset ledger, so a feature cannot be labeled *documented* without the source that documents it, and *generated* geometry must disclose how it was made.

The remainder proceeds as follows. Section 2 positions Technē against MCP tool use, the closed X3D and HAnim standards, Schema-Aligned Parsing and BAML, and prior LLM-driven Web3D authoring. Section 3 develops the epistēmē/technē gap through a catalog of four documented X3D silent-failure modes. Section 4 presents the man-in-the-middle architecture and its governing commitment to determinism. Sections 5 and 6 detail the two built mechanisms: in-transport argument repair with prescriptive correction, and the occupation gate. Section 7 reports the implementation, 147 passing unit tests, a controlled comparison against the bare server, and a live end-to-end smoke verification. Section 8 extends the discipline to a provenance bridge and discusses generalization beyond X3D and limitations.

2 BACKGROUND AND RELATED WORK

2.1 MCP and LLM tool use

The Model Context Protocol standardizes how a language model discovers and invokes external tools: a server advertises named operations with JSON-Schema-typed arguments, and the model drives them by emitting structured tool calls [1, 16]. The `x3d_mcp` server exposes an editing surface over an X3D scene—`create_node`, `set_field`, `add_child`, `add_route`—so the model operates a scene-graph editor rather than emitting a document in one shot [10]. Every mutation is a typed call on the wire, tractable to intercept. But a JSON-Schema argument type constrains only the *shape* of a call, not whether the edit is conventionally correct.

2.2 X3D 4.0 and HAnim

X3D 4.0 is an ISO declarative scene-graph standard with a formally specified, closed schema—an XSD and an accompanying object model [4, 6]. Its 4.0 revision adds the features most exposed to silent failure: `PhysicalMaterial` physically based rendering

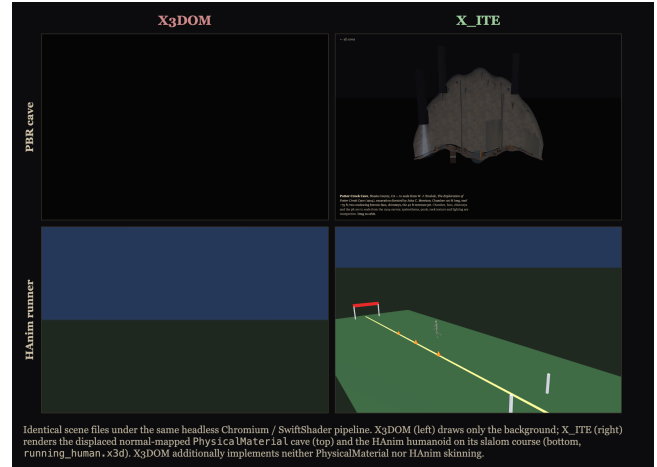


Figure 2: The renderer is a correctness property. The same scene under headless software WebGL: X3DOM (left) draws nothing for HAnim and X3D 4.0 `PhysicalMaterial`; X_ITE (right) renders both. An X3DOM-based gate would inspect a blank and pass it.

aligned with glTF [9], `EnvironmentLight` image-based lighting, and the articulated humanoids of HAnim [5]. Animation uses interpolator nodes driven through ROUTEs; node sharing uses DEF/USE. Critically, the same node type legally appears in several parent fields, distinguished only by `containerField`—a `HAnimJoint` as a skeleton root versus an ordinary children member; an `ImageTexture` as `baseTexture` versus the default texture. Which field belongs in which slot is craft the XSD does not express. Getting it wrong yields output that validates yet renders blank [10]. Conformant players—X_ITE [7], X3DOM [8], Castle [11]—faithfully render the wrong thing, because the document is, by the schema’s lights, correct. The X3D Examples Archive embodies this craft by convention, not enforcement [17].

2.3 Schema-Aligned Parsing and BAML

BAML productizes Schema-Aligned Parsing (SAP): deterministic, edit-distance repair of a model’s loosely structured output into a target schema, following Postel-style liberal acceptance [2, 3]. It pairs repair with validation as first-class constructs—`@assert` (blocking) and `@check` (advisory)—so craft predicates live in the schema rather than in scattered procedural checks. Technē builds on BAML, adopting both its repair engine and its assertion semantics. The difference is where the repair sits. BAML operates where application code calls a model (app→model); Technē inverts the direction, applying repair where a model calls a tool (model→tool), intercepting the arguments in transport before the real server sees them (Fig. 1). We inherit BAML’s constraints honestly: only block-level double-@@ assertions fire at runtime—field-level single-@ assertions are silently ignored, the very class of silent failure the layer exists to catch. So every Technē rule is named and mapped to a prescriptive correction that preserves the identity the catalog depends on.

2.4 LLM-driven X3D authoring

Prompt engineering for LLM-authored X3D has been studied directly: careful prompts measurably improve the rate at which a model emits usable scenes [13]. Adjacent work pursues accessibility, such as automated audio captioning [14]. Technē is complementary: rather than improving the prompt, it makes the craft hold mechanically *downstream* of whatever the model emitted. Comprehension is no longer load-bearing. The blackboard architecture [15]—staged cooperation among heterogeneous agents writing to a shared solution—provides a useful prior frame: Technē’s human/model/deterministic-layer split is a narrow modern instance, with the scene as the shared structure and the gate as control.

2.5 What is new

Against this background, the contributions listed in Sec. 1 are: the in-transport redirection of SAP onto tool-call arguments (inverting BAML’s `app→model` seam [1, 2]); documented craft compiled into prescriptive corrections [10]; the occupation gate; and the provenance bridge. None of these are provided by prior LLM-for-Web3D work [13]. The choice of renderer is itself a correctness property: X3DOM draws blank headless what X_ITE renders fully (Fig. 2).

3 THE CRAFT GAP: VALID BUT WRONG

An MCP server [1] exposes a domain as a flat surface of tools and node types; for X3D [4] authored through the Web3D `x3d_mcp` server every verb maps to a legal operation and every document checks against the X3D 4.0 XSD via the Unified Object Model [6]. As Sec. 1 framed it, schema conformance is nonetheless the wrong success criterion: a model emitting tool calls into this surface can produce a document that passes the XSD in full yet renders wrong, mis-lit, mis-animated, or blank—the validator reports success, the pixels report failure, and nothing in between throws. This is the gap between *epistēmē*, the decidable closed structure the XSD encodes, and *technē*, the contingent craft it does not: which `containerField` routes a node into which parent slot, which defaults must be stated rather than assumed, which lengths must agree, which order operations must occur in.

We catalog four concrete silent-failure modes, drawn from the `x3d.py` bug report [10], encountered while building HAnim humanoids and PBR scenes programmatically, serializing with `X3D(...).XML()`, and rendering in a conformant viewer. They share one signature, developed below: none throw, all validate, all fail only when a renderer runs and a human looks.

3.1 Dropped containerField

Every X3D node has a default `containerField`—the slot it lands in when none is stated. But many nodes legally appear in more than one parent field. A node placed in a non-default slot must name it explicitly, or a conformant reader files it into the default. The `x3d.py` serializer [10] emits such nodes without the attribute: a skeleton root meant for HAnimHumanoid.skeleton lands in children and the entire figure [5] renders invisible. The same defect drops PBR textures—an `ImageTexture` meant for `baseTexture` of a `PhysicalMaterial` [9] keeps its default texture slot, which the

material does not define, so a textured surface renders as flat base color.

3.2 EnvironmentLight.global

The X3D 4.0 lighting table specifies `SFBool global FALSE` for `EnvironmentLight`; `x3d.py` defaults it to `True` and then—because serializers omit fields equal to their internal default—drops `global` from the output. A spec-default reader parses the absent attribute as `false`, so image-based ambient lighting silently dies.

3.3 Interpolator key/keyValue mismatch

An interpolator’s `keyValue` array must hold exactly `len(key)` entries times the node’s per-key component count—four for `OrientationInterpolator`, three for `PositionInterpolator`, one for `ScalarInterpolator`. The XSD types both fields as float arrays and checks neither their relative length nor the multiple. A mismatch passes validation and silently corrupts the animation.

3.4 USE-before-DEF ordering

A USE reference must follow the DEF it names. Emit the USE first and the document is still well-formed XML and schema-valid, but the reference resolves to nothing and the reused geometry vanishes. This is an order-of-operations constraint that no per-element schema check can see.

These four modes share one property: each passes the XSD because the defect lives in container routing, a non-spec default, a cross-field length relation, or an ordering constraint—none of which the schema predicates over.

4 ARCHITECTURE

Technē is a man-in-the-middle for the Model Context Protocol [1]. The model connects not to the `x3d_mcp` server but to Technē, which launches the real server as a subprocess and connects to it as a client (Fig. 1); every `tools/call` passes through a decision layer before the upstream server runs it, and every result passes back through before the model sees it. To the model the surface is unchanged—the same tool names and arguments—except for a one-line hint on guarded tools that their arguments are repaired and validated in transit.

This transport position is the load-bearing design choice. BAML [2] applies Schema-Aligned Parsing where *code calls a model* (`app→model`), and Technē inverts the seam onto the arguments *a model emits into a tool* (`model→tool`). Nothing in the decision logic depends on the wire format, so the same core would front any MCP server; X3D via `x3d_mcp` [10] is the grounding domain, not a dependency.

4.1 Determinism

The design answers a specific failure of advice. Craft rules written into a server’s prose instructions—“set `containerField` explicitly”—are advisory: the model probably follows them, and the dice reset when a fresh session connects. A new model must re-comprehend the same instructions, and there is no guarantee it will. Technē makes the craft hold mechanically instead. The arguments pass *through* a layer that repairs and validates them deterministically, applying Postel-style liberal acceptance [3] to

what the model emits before the real server ever runs it. A correction discovered once becomes mechanism the next model cannot bypass. The corollary is deliberate: the runtime performs no per-call LLM round-trip. The validation is a deterministic Python mirror of the canonical BAML declarations in `baml_src/` [2], so a layer whose whole claim is determinism does not reintroduce a model into its own enforcement.

4.2 The per-call pipeline

For each call, `decide()` in `proxy.py` runs three steps. *First*, SAP-style repair (`repair.py`) coerces the raw arguments into conforming shape. The proxy sets `validate_input=False` so that Technē sees the model’s raw, possibly sloppy arguments before any upstream schema check can reject them—the repair layer cannot fix what the transport has already refused. *Second*, routing: a tool with a registered craft adapter (`create_node`, `add_child`, `def_node`, `use_node`, `set_field`, `add_route`) runs through it; a tool without one forwards unchanged—an opt-in stance that is a claim about the catalog, not an omission. If a tool has no adapter, there is no documented failure mode there. *Third*, the decision: the adapter returns a `CraftResult` that either rewrites and forwards a repairable argument set, or blocks the call and returns the prescriptive correction in its place. The call never reaches the real server until the arguments are clean.

The asymmetry is deliberate: a violation whose correct value is determinable is repaired silently and forwarded; one that requires a choice the layer cannot make is blocked and handed back to the model to redo.

4.3 Scene state

Some rules span more than one call: checking a child’s `containerField` needs the types of both child and parent, but `add_child` references nodes by opaque ID. Technē keeps a minimal `SceneState`—an `id`→`type` map, the DEF names seen, and the field values set—committed only on confirmed upstream success through a deferred `decide()/observe_result()` split detailed in Sec. 5.

4.4 What is not built

A cross-call automaton enforcing order-of-operations over a whole session is intentionally absent. Open-ended authoring should resist a straitjacket. The likely truth is a partial order—a few hard edges in a large open space—so the next step is instrumentation: log the verb order of clean renders, let accumulated usage identify real constraints before committing an automaton.

5 REPAIRING AND VALIDATING TOOL-CALL ARGUMENTS

This is the layer between the model’s emitted tools/call and the real server. It decides, per call, whether the arguments may be forwarded as written, rewritten and forwarded, or refused with a correction (Fig. 1). It runs in two stages: a deterministic repair of argument shape (`repair.py`), followed by craft rules over the cleaned arguments (`craft.py`, `rules.py`). Both are LLM-free Python, honoring the determinism commitment.

5.1 SAP-style argument repair

The first stage applies Postel’s robustness principle—“be liberal in what you accept” [3]—to the tool-call arguments, the same move BAML productizes as SAP for model output [2], here redirected onto arguments a model emits *into* a tool. `sap_repair` is deliberately conservative: it unwraps and coerces shape but never invents content. It strips markdown fences and any chain-of-thought preamble preceding the first JSON object. It un-stringifies values under structured keys (`fields`, `value`, `arguments`, `args`) that arrived as embedded JSON text—a common model output artifact when the model wraps a JSON dict in quotes. It coerces obvious scalar leaves—`"true"/"false"` to `bool`, numeric strings to numbers—only where a boolean or numeric field is expected (e.g., a global leaf inside `fields`). It never raises: worst case it returns the original dict or `{}`, so the caller can always forward. This is the conformance-shaping layer; the semantic rules run only on its output. When the generated BAML client is present, `b.parse` substitutes the full edit-distance engine for this dependency-free baseline.

5.2 The rule-to-fix catalog

The second stage is the documented craft. The catalog of X3D silent-failure modes—constructs that pass the XSD yet render wrong or blank—is recast as a table `{rule: (severity, prescriptive_correction, source)}` in `rules.py`. Each entry names a single rule and maps it to a correction string. The whole point is the difference between “rejected” and “rejected: use `containerField='baseTexture'`.” A thrown error reports only that some predicate failed; it says nothing about the fix. By naming every rule (`container_field_required`, `interp_lengths_match`, `use_after_def`, ...) and binding it to a templated correction carrying the offending types, fields, and legal alternatives, the error message *is* the fix. Each entry cites its source—most to the `x3d.py` bug report [10]—so the craft is traceable. Severity is binary: HARD blocks (BAML @@assert); SOFT advises (@@check).

5.3 Two outcomes: repair or block

A craft rule resolves to one of two outcomes, and the discriminator is whether the correct value is determinable without guessing intent. *Repair* fires when exactly one legal value exists: a `containerField` for a placement with a single legal slot has one answer. Technē rewrites the argument, records a note, and forwards the corrected call, so the craft holds mechanically whether or not the model understood it. *Block-and-correct* fires when the fix needs a choice the layer cannot make: which of the five PBR texture slots (`baseTexture`, `normalTexture`, etc.) a texture was meant for; which DEF a dangling USE intended; or which of `key/keyValue` is the wrong length when their parity fails. In these cases Technē refuses to forward and returns the prescriptive correction listing the legal alternatives, leaving the intent decision to the model on retry. Filling in the choice would be inventing content—exactly what the repair stage forbids. The outcomes compose: `CraftResult` carries repaired arguments, a corrections list (any non-empty list blocks the forward), notes, and the applied rule names, and merge folds several single-rule results into one.

5.4 Worked example: the texture containerField

A model places an `ImageTexture` under a `PhysicalMaterial` via `add_child` with no `containerField`. The XSD is satisfied. But the default texture slot is one a PBR material [9] does not define, and the surface renders untextured. Technē looks up both types in its scene state, finds five legal slots in `MATERIAL_TEXTURE_SLOTS`, and—because more than one is legal—does not guess. It blocks and returns `container_field_required`, naming the child, the parent, the silently-dropped default, and the five slots. Had the parent been an `HAnimHumanoid` [5] with an `HAnimSegment` child, whose only non-default slot is `segments`, the same machinery would repair—set `containerField='segments'` and forward—because the answer is unique. One mechanism, two outcomes, decided by slot cardinality.

5.5 The stateful proxy

The `containerField` rule needs the child and parent *types*, but `add_child` references nodes by *id* [6], learned only from `create_node` responses, so `TechnēProxy` keeps the minimal `SceneState` of Sec. 4 (`id_to_type`, `node_fields`, `defined_defs`, `def_name_to_type`). Its correctness hinges on deferred commit: `decide()` only reads state and records what it *would* commit; `observe_result()` commits the `id→type`, `DEF`, and field updates only after a confirmed-successful upstream response—so a failed `create_node` registers no type, and a premature `DEF` from a failed `def_node` cannot satisfy a later `USE`. When the proxy cannot resolve a type, it degrades to a non-blocking note rather than a false correction.

5.6 The BAML mirror

`baml_src/craft.baml` declares the same modes as block-level `@@assert/@@check` over classes (`TextureAttachment`, `EnvironmentLightArgs`, `InterpolatorArgs`, `UseRef`). It is the source of truth and the SAP path when wired; the Python engine is its deterministic, dependency-free mirror. One trap deserves mention: only the double-`@@` block form fires at runtime. Field-level single-`@` assertions are silently ignored—the very class of silent failure the layer exists to catch.

5.7 The EnvironmentLight.global boundary

The `EnvironmentLight.global` case (Sec. 3) marks the layer's honest boundary. Because the divergence lives below the wire—`x3d.py` defaults `global` to `True` internally, omits it on serialization, and rejects a `global` keyword argument (it uses `global_`)—the argument layer *cannot* repair it. So Technē demotes it to a `SOFT` advisory here and pushes the hard fix into the serialization step at the gate, which injects `global='true'` into the XML directly via `reassert_envlight_global`. The unit tests explicitly assert that `global` is *not* injected into the arguments.

5.8 Edit verbs and standing semantics

Two extensions carry the same discipline past the `create_node/add_child` surface. First, the content-edit verbs—`modify_x3d_node`, `move_x3d_node`, `remove_x3d_node`, `convert_x3d`—mutate an existing document and report success without re-checking it. Technē post-validates their results: an XSD post-pass catches a misspelled attribute a `modify` wrote, a semantic post-pass re-checks the `containerField` a `move` never revisited, and an element-count diff flags nodes a `convert` silently dropped—faults no single validator reports on its own. Second, a standing-semantics layer (`semantics.py`) surfaces the invariants a session forgets—X3D is right-handed and in meters and radians; a `TimeSensor` animates only through `ROUTEs`; an `HAnimJoint`'s rest pose is identity; PBR materials are not automatically lit by the headlight alone. These ride back on the relevant tool result as soft reminders, de-duplicated with a 30-call cooldown before re-firing the same key, capped at two reminders per call. A prompt read once at the top of a session would have lapsed; these persist.

6 THE OCCUPATION GATE: RENDER AND LOOK

The two preceding mechanisms operate on parsed values: SAP repair rewrites arguments; craft rules predicate over the cleaned arguments and a scene-state ledger. Both are functions from a structure to a verdict. This is also the ceiling of an assertion language. A BAML `@@assert` [2], like a JSON Schema keyword [16] or an XSD facet, can only quantify over the term in front of it. It cannot leave the term, execute a renderer, get pixels back, and check that those pixels are non-blank. Yet that is exactly the test the silent-failure modes demand: a dropped `containerField` or a wrong `EnvironmentLight.global` produces a document that satisfies every predicate you can write over it and still renders to an empty frame. The verdict that matters is not a property of the document but of its image. The occupation gate crosses that boundary: it executes a renderer and inspects the pixels—*verify-by-render*—the part for which there is no assertion-language equivalent, because it is side-effecting by construction.

6.1 The gate's logic

The gate fires when a model declares the artifact done and runs three stages (`gate.py`). First, the semantic validator; if the document is not clean, it blocks and returns the errors as corrections, so a structurally dead scene never reaches the renderer. Second, it renders the scene and reduces the PNG to a single luminance figure—the standard deviation across pixels—treating anything below a small threshold (`BLANK_STDDEV = 3.0` grayscale levels) as blank. This is the failure only a render catches, so the correction is specific: check that geometry is present, lit, and on-camera; check that `HAnim` and PBR `containerFields` are correct. Third, once the frame is non-blank, the gate applies sign-off proportional to what is downstream.

A design commitment runs through the blank check: fail safe. If the gate cannot decode the image, it treats it as blank and blocks. A false block costs a retry; a false pass ships a blank artifact. The second failure is the one nothing else catches.

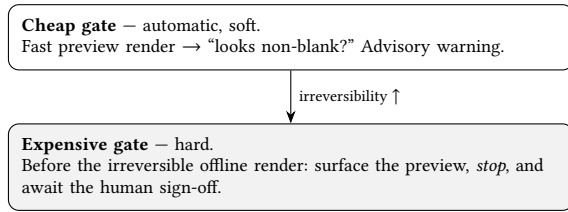


Figure 3: The occupation gate is graduated by irreversibility: a soft check before cheap reversible work, a hard human sign-off before expensive irreversible work.

6.2 The renderer is a correctness property

The gate is only as honest as its eyes. Under headless software WebGL, X3DOM renders blank for exactly the content this project showcases—HAnim humanoids and X3D 4.0 PhysicalMaterial surfaces—while X_ITE renders both [7–9, 12] (Fig. 2). A gate wired to X3DOM would read a correct skeleton as a blank and either block every valid figure or, with the polarity flipped, certify blanks as fine. Its verdict would be a property of the renderer, not the scene. The gate requires X_ITE: for an adjudicator that decides by looking, the renderer that can actually display the content is the only one that tells the truth.

6.3 Graduation by irreversibility

The sign-off is graduated by how irreversible the work it guards is (Fig. 3); the frame is that of an architect directing a draftsman, with the human signing off and the model never committing irreversible work on its own authority. A cheap, fast preview render is reversible, so the gate before it is soft and automatic, wired directly into the transport (server.py): on render verbs (render_image, render_current_scene) the proxy inspects the returned image and, if blank, appends an occupation-gate warning to the relayed result—in-band, on every render, at no extra round-trip. The expensive, long-running offline photoreal render is irreversible, and the gate before it is hard: when the cost is marked expensive and the human sign-off has not returned approval, the gate surfaces the validated preview and stops (awaiting_architect); its default sign-off returns false, so expensive work always waits. This hard gate is host-driven—the upstream server exposes no offline-render verb yet, so it lives in the OccupationGate library awaiting one, while the cheap gate already runs in transport. The discipline scales with the stakes: a glance before reversible drafting, a human signature before irreversible building.

7 IMPLEMENTATION AND EVALUATION

Technē is a Python package that sits in front of the Web3D x3d_mcp server [10] as a man-in-the-middle MCP proxy (Fig. 1). It is deliberately modest in size and dependencies: the claim is mechanistic, so the runtime that enforces it is the artifact under test.

7.1 Implementation

The package follows the three insertion points of Sec. 4. rules.py is the rule-to-fix catalog. craft.py implements the four modes of Sec. 3 through the repair/block outcomes of Sec. 5; repair.py is

the dependency-free SAP-style repair. proxy.py is the stateful decision core with its decide()/observe_result() split. gate.py is the occupation gate (Fig. 3), server.py the MCP transport, and baml_src/ the canonical BAML declaration of the same rules as @@assert/@@check. semantics.py carries the standing invariants; provenance.py the opt-in provenance gate with ledger checks; config.py the per-deployment profiles; and eval/ the scripted-and-live evaluation harness.

The runtime depends only on the mcp SDK for the stdio transport, Pillow for the gate’s pixel inspection, and the standard library; the craft and repair engines are dependency-free, so Technē runs without a per-call LLM round-trip (the determinism commitment, Sec. 4). Because x3d_mcp is itself a Python MCP server, proxy and upstream co-locate in one process tree—python -m technē.server -- python ../src/server.py launches the real server as a subprocess the model reaches only through Technē: list_tools forwards the upstream surface (tagging guarded tools with a hint that their arguments are repaired in transit), and call_tool runs the decision core, answering a hard violation with its prescriptive correction and forwarding repaired arguments otherwise. For the edit verbs, the transport post-validates the returned document through the upstream’s own validate_x3d and validate_semantic; for autofix_x3d it applies reassert_envlight_global to the returned XML; and for render verbs it runs the blank detector on the returned image. The provenance gate fires optionally on edit and render results when the profile enables it.

7.2 Unit tests

The package carries 147 unit tests across eleven files. The craft tests exercise each documented mode in both directions: a missing or wrong containerField repaired to the correct slot while a correctly-placed node is left alone; a mismatched interpolator key/keyValue blocked with a correction naming the expected and actual lengths; a USE with no prior DEF blocked; a non-image texture URL warned without blocking; empty, unseparated, or out-of-range coordIndex caught. The proxy tests cover the stateful surface a stateless validator cannot: placement checked against the registered node type, single-slot repair versus multi-slot block (where the right answer cannot be guessed), geometry-health checks on IndexedFaceSet via both create_node and set_field, USE-reference type inheritance, and failed creates or DEFs that must *not* register so a later USE still blocks. The SAP-repair tests confirm the unwrapping and never-raises contract. The gate tests verify blank-versus-non-blank discrimination, the cheap/expensive graduation, and that the gate recognizes the *actual* validate_semantic string the real server emits, not a mock’s. The transport tests confirm guarded-tool tagging, that a blocked call never forwards while still returning its correction, that the EnvironmentLight advisory is relayed without injecting a breaking argument, and that the blank-render warning is appended. Further suites cover the edit-verb post-pass (a misspelled modify, an un-rechecked move, a node-dropping convert), the standing-semantics reminders with cooldown and re-arm, the provenance gate at L1–L2, and the Blender craft catalog’s

19 checks with 25 tests of their own (zero false positives on a clean default scene).

7.3 Controlled comparison

The layer’s value is the difference between driving a task through the bare server and through Technē, so we measure it directly. Six short authoring sequences (eval/) run through x3d_mcp twice—once raw, once behind Technē, in isolated processes. Five carry a documented silent-failure mode (a texture missing its containerField; a key/keyValue length mismatch; a USE before its DEF; a duplicate DEF; a ROUTE with no DEF’d endpoint) and one is a correct control. Run raw, two of the five pass *silently*—the dropped texture (Fig. 4, bottom) and the corrupted interpolator both validate and return success, with no indication that anything is wrong—while three are rejected *loudly* with a bare error that names no remedy, leaving the model to guess. Behind Technē all five are caught: the silent ones blocked or repaired before they land, the loud ones turned into a prescriptive correction naming the exact fix, every block carrying the rule name and the legal alternatives. On the correct control, Technē is invisible—no blocks, no added round-trips, no false positives. The measurement is over documented modes, not a population, but it makes the thesis concrete: silent and unhelpful failures become caught, named ones, at no cost when the draft is already right.

7.4 Live end-to-end smoke

The decisive test is a live smoke (smoke_live.py) in which a real MCP client drives create_node(PhysicalMaterial), create_node(ImageTexture), an add_child with a missing containerField, create_node(EnvironmentLight), and a render, through Technē fronting the genuine x3d_mcp. It confirmed the system end to end: guarded-tool tagging, the add_child block with the baseTexture correction, and the blank-gate firing on the render.

The live integration caught three faults the unit tests could not, because each lived in the seam between Technē and the real server. First, the proxy initially dropped the upstream’s structuredContent; real FastMCP tools declare an outputSchema [16], so content-only relays failed. The fix was to return a CallToolResult directly, relaying structuredContent untouched. Second, the proxy had to disable upstream input validation so that SAP repair sees the model’s raw arguments rather than pre-rejected ones—the repair layer cannot fix what the transport has already refused. Third, the EnvironmentLight.global repair was mis-targeted: this divergence is a serialization-layer problem the argument layer cannot reach, because x3d.py rejects a global keyword argument. The fix was downgraded from an injected argument to an accurate advisory.

The lesson is itself an instance of the paper’s thesis: integration against the real server surfaced craft—the output-schema contract, the validation ordering, the serializer’s true argument surface—that the unit harness had mocked away. The contingent knowledge of making the thing come out right is not deducible from the schema; it is discovered at the seam. What we demonstrate

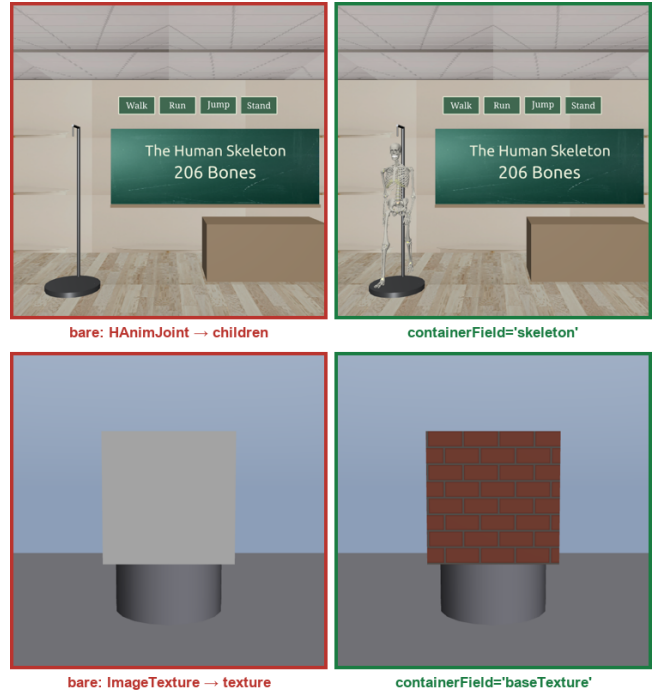


Figure 4: Two silent failures the XSD admits. Each pair is the same scene differing in a single containerField: *left* (red) the bare default, *right* (green) the value Technē enforces. *Top*: an HAnim skeleton root defaulted into children—the figure vanishes, leaving an empty stand beneath a board reading “206 Bones.” *Bottom*: an ImageTexture defaulted into texture, a field PhysicalMaterial does not define—the brick vanishes. Both pass validate_x3d. Both reproduce in a second independent runtime (Castle Model Viewer 5.3 [11]) as well as in X_ITE [7], confirming the fault is format-level.

is that the mechanism exists, runs, and holds; its scope—the absence of a user study or corpus benchmark over LLM-authored scenes [13, 17]—is stated in Sec. 8.

7.5 Deployment against real scenes

We ran two unrelated real scenes through the tooling—an agent-simulation Script-driven X3D scene and a 150-joint LOA5 HAnim classroom skeleton. Both render correctly under X_ITE, yet validate_semantic flagged eight errors on the HAnim scene, every one false: seven valid ROUTEs into a Script’s dynamic fields and a correctly-placed MetadataSet. The cause: Script and ProtoInstance carry dynamic interfaces (fields declared in the scene, not the object model), and metadata nodes implement an abstract interface (X3DMetadataObject) that X3DUOM’s type chain omits [6]. We fixed both resolutions—contributing the ROUTE fix upstream—after which the scene validates clean. The scene was correct all along; the tooling was not.

8 DISCUSSION

8.1 Documented craft as mechanism

Technē’s contribution is not the discovery of X3D craft but its conversion into enforced mechanism. The silent-failure modes it guards against are documented in the `x3d.py` bug report [10]; Technē transcribes their stated workarounds into the rule-to-fix map so the craft holds mechanically, where prose instructions are merely advisory and reset when a fresh session connects [2]. With the catalog fixed, the layer’s only open-ended job is to surface the *next* undocumented mode—the job of the planned Point-1 instrumentation, which would log clean-render verb order and let accumulated usage decide whether a cross-call ordering constraint is real, rather than legislating a total order in advance.

8.2 The provenance bridge

The same move—documented discipline compiled into a deterministic check—reaches past correctness to a second concern: honesty. A reconstruction authored at machine speed invites a particular risk: geometry that looks authoritative while accountable to no source, AI imagery presented as documentary record. Technē answers with an opt-in provenance gate (`provenance.py`).

The convention is a standardized X3D provenance `MetadataSet` carrying: `provenance` (documented/interpretive/generated), `catalogId`, `sourceCitation`, `publicDomain`, and `generationMethod`. The gate checks each block against an `archive.json`-shaped asset ledger and is graduated like the occupation gate. *L1 disclosure* keeps the tags well-formed and requires that generated geometry name its `generationMethod`, so synthetic content is never mistaken for record—the universal, ledger-free rung, in the spirit of content-provenance credentials. *L2 sourcing* demands that a documented claim resolve to a cited, public-domain ledger entry: you cannot label a feature *documented* until you hold the source that documents it, and no uncleared or AI-made asset may be presented as documentary. A content-level *L3—must-not-invent* rules over taxon, depth, calibrated date—is specified but deliberately unbuilt, pending a structured claim schema.

The gate is off by default. It nags no one who has not opted in, activated through the `TECHNE_PROVENANCE_LEVEL` environment variable and the profile system (`config.py`). Its rules live apart from the craft catalog to mark provenance as an extension, not core craft. The bridge is the paper’s thesis turned on the artifact itself: the documented/interpretive boundary that a careful reconstruction draws by convention becomes a boundary the layer holds by mechanism.

8.3 Generalization beyond X3D

The claim that the pattern is not specific to X3D now rests on concrete evidence. A working Blender craft layer applies the same three moves to BlenderMCP, a second MCP server whose tool surface is structurally opposite to that of `x3d_mcp`. Where the X3D proxy intercepts individually typed tool calls—`create_node`, `set_field`, `add_child`—and validates their arguments before execution (pre-validation on the wire), BlenderMCP exposes a single `execute_blender_code` tool that accepts arbitrary Python; pre-validating opaque code is not tractable. The Blender layer

therefore shifts the seam: it inspects the live scene graph *after* execution, querying Blender’s object model through the addon’s socket for the same class of silent-failure patterns. Its catalog comprises 19 checks across four domains—Material (e.g. a wired but imageless texture node, a shader whose alpha is below unity yet `blend_method` remains `OPAQUE`), Geometry (flipped normals, empty meshes), Scene (missing camera or lights), and Modifier (a Boolean with no operand, an Array with zero offset)—each returning the same HARD/SOFT severity and the same prescriptive-message-is-the-fix contract as the X3D rules. On Blender’s default scene the inspector produces zero findings, reproducing the “invisible on correct input” property (Sec. 4) without tuning.

What transferred verbatim was everything except the checks themselves: the inspection pattern, the severity semantics, and the contract that the error message names the fix. What changed was the interception point—pre-validation on the wire versus post-validation on the scene graph. The two implementations thus span the structured/unstructured spectrum of MCP tool surfaces [1]: individually typed operations whose arguments can be repaired in transit at one end, an opaque code-execution tool whose effects can only be judged after the fact at the other. That the same discipline holds at both ends—with the rule catalog as the sole domain-specific asset—suggests the pattern generalizes across tool-surface designs, not only across content domains within one design.

8.4 Limitations

Coverage is opt-in per tool: the layer’s reach is exactly as wide as its catalog of registered adapters. An undocumented mode passes silently until instrumentation surfaces it. The hard, expensive gate is specified and wired in `gate.py`, but the offline render verb it gates on awaits an upstream addition; today the soft “looks non-blank?” gate runs against `X_ITE` [7], while the X3DOM path [8] returns blank for the HAnim humanoids this project showcases. The BAML code-generation path—calling `b.parse` for SAP—is deferred in favor of the deterministic Python mirror. The Blender implementation demonstrates generalization (Sec. 8) but does not yet integrate a render-and-look gate of its own. We report 147 passing tests, a controlled comparison over documented modes, and a live smoke; no user study or large-corpus benchmark yet establishes how often the layer catches real model errors across a population, and the content-level provenance tier (L3) and the cross-call automaton are deliberately unbuilt.

8.5 Ethics

The human is the author and decision-maker who holds intent and signs off. The model is a tool, used under direction, that originates no intent and authorizes no irreversible build. Technē carries what the architect composed without silently corrupting it and holds the gate so the sign-off stays human. The layer makes the tool safer to use; it does not make the tool an author.

9 CONCLUSION

A schema is the wrong success criterion for artifacts a language model authors through a tool surface. It checks structure—*epistēmē*—but not craft—*technē*—and the two part company

exactly where it matters: where a model emits tool calls that pass the XSD and still render wrong or blank. Technē closes that gap on the MCP transport through three built mechanisms—in-transport argument repair, prescriptive corrections from a documented catalog, and an occupation gate that verifies by rendering—and extends the same discipline to content honesty in an opt-in provenance gate. None is specific to X3D.

What remains is evidence at scale, and the discipline to keep the human the one who holds intent and signs off, with the model as the draftsman and the layer as the gate.

USE OF GENERATIVE AI

Anthropic’s Claude (Claude Code) was used as a tool under the author’s direction for implementation, validation, render automation, and drafting. Per the ACM Policy on Authorship the generative AI is not an author: the author conceived the work, made every substantive decision, and is solely responsible for the content.

REFERENCES

- [1] Anthropic. 2024. *Model Context Protocol Specification*. <https://modelcontextprotocol.io>
- [2] BoundaryML. 2024. *BAML and Schema-Aligned Parsing*. <https://github.com/BoundaryML/baml>
- [3] J. Postel. 1980. *Transmission Control Protocol*. RFC 761. (The robustness principle: be conservative in what you do, liberal in what you accept.)
- [4] ISO/IEC 19775-1:2023. *Extensible 3D (X3D) Part 1: Architecture and base components* (X3D 4.0). Web3D Consortium / ISO.
- [5] ISO/IEC 19774:2019. *Humanoid Animation (HAnim)*.
- [6] Web3D Consortium. *X3D Unified Object Model (X3DUOM)*. <https://www.web3d.org/specifications/X3DUOM.html>
- [7] H. Seelig. *X_ITE: An X3D Browser for the Web*. https://create3000.github.io/x_ite/
- [8] J. Behr, P. Eschler, Y. Jung, and M. Zöllner. 2009. X3DOM: a DOM-based HTML5/X3D integration model. In *Proc. 14th Int. ACM Conf. on 3D Web Technology (Web3D '09)*. doi:10.1145/1559764.1559784
- [9] Khronos Group. *glTF 2.0 Specification* (metallic-roughness physically based rendering).
- [10] Web3D Consortium. *x3d.py: the X3D Python (SAI) package*. <https://pypi.org/project/x3d/>
- [11] M. Kamburelis. *Castle Game Engine / Castle Model Viewer*. <https://castle-engine.io/>
- [12] Microsoft. *Playwright*. <https://playwright.dev/>
- [13] N. Polys, A. Mohammed, and B. Sandbrook. 2024. Prompt Engineering for X3D Object Creation with LLMs. In *Proc. 29th Int. ACM Conf. on 3D Web Technology (Web3D '24)*. doi:10.1145/3665318.3677159
- [14] N. Polys and S. M. Wasi. 2023. Increasing Web3D Accessibility with Audio Captioning. In *Proc. 28th Int. ACM Conf. on 3D Web Technology (Web3D '23)*, Article 6, 1–10. doi:10.1145/3611314.3615902
- [15] H. P. Nii. 1986. Blackboard Systems. *AI Magazine* 7, 2 (1986), 38–53.
- [16] *JSON Schema*. <https://json-schema.org/>
- [17] Web3D Consortium. *X3D Examples Archives*. <https://www.web3d.org/x3d/content/examples/>